CRONet HLS Steps 4-6: Validation, Static Weights, and PL Kernel Architecture

Detailed implementation note for the 60x20 FP16 Vitis HLS CRONet build, with explicit alignment to the CRONet network specification and the AMD Versal AIE-ML paper requirements.

Scope. This document expands the implementation-flow steps covering HLS half layerwise validation, static FP16 weight ROM generation, and the single-kernel PL architecture. It focuses on whether these pieces satisfy the CRONet 1:1 network contract and the AIE-paper benchmark requirements: full inference on the accelerator, persistent on-chip weights, on-chip intermediate activations, and external memory used only for input injection and final output collection.

Precision note. The AIE paper selects BF16 for the Versal AIE-ML implementation. This HLS repository intentionally implements the current exported vector package in IEEE FP16 / binary16. Therefore the correct comparison label is AIE BF16 vs PL/HLS FP16, unless both paths are rebuilt with the same datatype.

Checkpoint	Current status / evidence
HLS half layerwise validation	Vitis HLS half debug path compares named FP16 layer outputs against exported golden tensors; expected flow passes sanity, ramp, sparse, and random cases.
Static weight storage	Generated header include/generated/cronet_weights_fp16_half.hpp stores all weights/biases as static const fp16_t arrays included in the synthesized kernel.
PL kernel architecture	src/cronet_kernel.cpp exposes only branch_input, trunk_input, and output as AXI memory ports; weights and intermediate tensors are internal kernel storage.
Hardware build status	Vitis 2025.2 produced a self-contained VEK280 hardware SD package; timing constraints were reported as met in the routed implementation.

Primary local references used. CRONet_HLS_1to1_Network_Spec_FP16.md; ref_sources/croNET_AIE-ML ENGINE.pdf; ref_sources/croNET.pdf; vitis-hls-CROnet/src/cronet_kernel.cpp; vitis-hls-CROnet/src/cronet_half_debug.cpp; vitis-hls-CROnet/tests/tb_half_layerwise.cpp; vitis-hls-CROnet/include/generated/cronet_weights_fp16_half.hpp.

AIE-Paper Requirements Translated to PL/HLS

The AIE paper is useful here because it describes the benchmark philosophy, not only an AIE-specific implementation. Its central requirement is that CRONet inference should be realized as an accelerator-resident dataflow pipeline, with off-chip traffic reduced to the model boundary. The PL/HLS implementation mirrors that requirement using FPGA kernel storage rather than AIE local memory and Memory Tiles.

Requirement from CRONet/AIE context	Meaning for this PL/HLS implementation	Implementation status
Full CRONet inference on accelerator	BranchNet, TrunkNet, and final multiply-plus-bias must run inside the Vitis kernel rather than as host-side layers.	Satisfied by one top-level cronet_kernel containing the full branch path, trunk path, and final combine.
Persistent on-chip weights	Weights should not be streamed from DDR every inference. They should be compiled or initialized into accelerator-local storage.	Satisfied by static const fp16_t generated arrays included by the kernel.
On-chip intermediate activations	Layer outputs may be materialized for debug, but they must remain inside accelerator resources rather than external DDR.	Satisfied by local/static fp16_t arrays inside cronet_kernel; no m_axi ports exist for intermediates.
DRAM used only at boundary	External memory traffic is allowed for reading branch/trunk inputs and writing final output.	Satisfied by only three m_axi arguments: branch_input, trunk_input, output.
Report precision honestly	AIE paper uses BF16; this HLS revision uses FP16 because the available vector package is FP16.	Satisfied if results are labeled PL/HLS FP16 and not presented as same-datatype BF16.

The original CRONet paper defines the model as a deep operator-style network with branch and trunk subnetworks. The branch network processes material-density history and includes recurrent temporal modeling; the trunk network processes load/location information; the outputs are combined to produce the displacement field. The HLS kernel follows that structure directly: branch path, trunk path, then elementwise branch * trunk + learned scalar bias over the [1,21,61,2] output.

The AIE paper further emphasizes the latency reason behind the storage rule: off-chip movement during inference is expensive. The HLS implementation therefore treats DDR as a host boundary and not as a layer-to-layer tensor store. This is the key architectural match between the PL/HLS design and the AIE paper, even though the compute fabric is different.

Step 4: HLS Half Layerwise Validation

Purpose

The HLS half checkpoint is the first validation point that uses the same HLS-facing datatype as the kernel implementation. Earlier native tests prove the graph, indexing, and golden-vector package using normal software types. Step 4 asks a narrower and more hardware-relevant question: when the network is expressed with Vitis HLS half, do the named tensors still match the exported FP16 golden values?

This is intentionally layerwise. A final-output-only check can hide errors that cancel or appear late. Layerwise comparison finds the first failing semantic boundary: convolution padding, SiLU, max pooling, adaptive average pooling, RNN recurrence, linear weight layout, final reshape, or final bias.

Mechanics

- The HLS type is selected through include/cronet_half_type.hpp. Under Vitis HLS, fp16_t resolves to half from hls_half.h; on the Windows/native smoke path it can resolve to float only to catch C++ integration mistakes.
- src/cronet_half_debug.cpp implements the complete CRONet forward path using fp16_t and stores named intermediate tensors in ActivationsHalfDebug.
- tests/tb_half_layerwise.cpp loads branch/trunk FP16 inputs from data/cronet_hls_fp16_vectors and compares the generated activations against golden/fp16_torch_f32.

- The Vitis script vitis/half_layerwise_csim.tcl runs the checkpoint inside Vitis HLS so the actual HLS half type and HLS math headers are exercised.

Named validation coverage

Area	What the checkpoint protects
Input reshape and row-major layout	Confirms branch/trunk binary files are interpreted in the same C row-major order as the export package.
Branch Conv2D + SiLU	Confirms 3x3 padding=1, per-channel bias, FP16 tensor storage, and activation ordering.
Branch MaxPool + AdaptiveAvgPool2D	Confirms the 2x2 stride-2 pool and per-history/channel average to [10,32,1,1].
Branch RNN	Confirms sequence length 10, hidden size 64, tanh recurrence, zero initial state, and use of both bias_ih_l0 and bias_hh_l0.
Branch FC path	Confirms FC1, SiLU, and FC2 use weight[out,in] layout and produce 2562 branch coefficients.
Trunk Conv3D + SiLU	Confirms 3x3x2 kernels, PyTorch-style padding behavior, and the W dimension growth from 2 to 3 then 4.
Trunk AdaptiveAvgPool3D + FC path	Confirms PyTorch floor/ceil windowing into [1,32,15,10,1] and final 2562 trunk coefficients.
Final combine	Confirms final_mul_pre_bias and final_output use branch * trunk + scalar out_bias over [1,21,61,2].

Why this matches the specification and AIE paper

The CRONet HLS spec requires layer-by-layer comparison before full-system comparison and requires the half-compute path to match fp16_torch or fp16_torch_f32 within the declared tolerance. Step 4 is exactly that gate. It also mirrors the AIE paper in operator coverage: the design includes 2D/3D convolution, GEMM/linear layers, max pooling, adaptive pooling, nonlinear activation, RNN, and final elementwise fusion.

The precision caveat is deliberate: this validates FP16 HLS behavior, whereas the AIE paper selected BF16 for its final AIE inference path. The validation therefore proves compliance with the current FP16 HLS export package, not same-datatype equivalence with the AIE implementation.

Step 5: Static FP16 Weight ROM Generation

Purpose

The AIE paper repeatedly treats weight movement as a major performance risk and stores compute-layer weights persistently inside the AIE array. The PL/HLS equivalent is to compile weights into the FPGA kernel as static constant arrays so Vitis maps them to local ROM-like resources such as LUTROM, BRAM, or URAM. This removes runtime weight transfer from DDR.

Generation flow

```
data/cronet_hls_fp16_vectors/weights/fp16_words
-> scripts/gen_fp16_half_header.py
-> include/generated/cronet_weights_fp16_half.hpp
-> #include in src/cronet_kernel.cpp and src/cronet_half_debug.cpp
```

The repository also contains a raw-word header path for ap_uint<16>/uint16_t storage experiments. The active correctness-first kernel uses the direct half header so arithmetic boundaries are explicit and easy to inspect.

Weight group	FP16 scalar count	Storage intent
Branch Conv2D layers	4,800	Small ROM/BRAM/LUTROM group.
Branch RNN weights and biases	6,272	Persistent recurrent parameters; both bias vectors retained.
Branch FC1 + FC2	107,642	FC2 dominates branch parameters; likely BRAM/URAM after synthesis.
Trunk Conv3D layers	9,552	Small-to-medium convolution ROM group.
Trunk FC1 + FC2	297,082	Largest weight region; major URAM/BRAM consumer.
Output scalar bias	1	Broadcast scalar bias for final output.
Total	425,349 FP16 values (~0.81 MiB)	Fits in FPGA on-chip memory budget for the correctness-first build.

Implementation evidence

include/generated/cronet_weights_fp16_half.hpp declares each parameter group as static const fp16_t. Representative arrays include branch_conv1_weight, branch_rnn_weight_ih_l0, branch_rnn_weight_hh_l0, branch_fc2_weight, trunk_conv1_weight, trunk_fc1_weight, trunk_fc2_weight, and out_bias. Because these arrays are not function arguments and are not attached to m_axi interfaces, they are not runtime DDR buffers.

The current routed HLS estimate shows the expected footprint pattern for an on-chip weight design: URAM is the tightest resource class, followed by BRAM/LUT/DSP. The reported baseline was approximately BRAM_18K 32%, DSP 29%, FF 7%, LUT 32%, and URAM 51%. That resource mix is consistent with large FC weights being held on chip rather than streamed externally.

Why this matches the specification and AIE paper

The CRONet HLS spec explicitly requires weights to be stored on chip as static arrays or initialized BRAM/URAM ROMs. The AIE paper uses AIE local memory and Memory Tiles for the same architectural reason: avoid repeated off-chip weight fetching. The PL/HLS implementation matches this requirement through FPGA-local ROM-style storage. The storage technology differs, but the data-movement contract is the same.

Step 6: Single PL Kernel Architecture

Top-level boundary

The deployed kernel boundary is intentionally small. src/cronet_kernel.cpp exposes only three data buffers to the host: branch_input, trunk_input, and output. The AXI memory interfaces are therefore boundary-only interfaces, not layer

interfaces.

```
extern "C" void cronet_kernel(
const cronet::fp16_t* branch_input,
const cronet::fp16_t* trunk_input,
cronet::fp16_t* output);

#pragma HLS INTERFACE m_axi port=branch_input ... depth=12000
#pragma HLS INTERFACE m_axi port=trunk_input ... depth=2562
#pragma HLS INTERFACE m_axi port=output ... depth=2562
```

Internal dataflow

Path	Implemented layers	Spec relationship
BranchNet	Conv2D -> SiLU -> Conv2D -> SiLU -> MaxPool2D -> AdaptiveAvgPool2D -> RNN -> FC1 -> SiLU -> FC2	Matches CRONet branch processing of the 10-frame material-density history and temporal RNN component.
TrunkNet	Conv3D -> SiLU -> Conv3D -> SiLU -> AdaptiveAvgPool3D -> FC1 -> SiLU -> FC2	Matches CRONet trunk processing of nodal load/location tensor into 2562 trunk coefficients.
Final combine	branch_fc2[i] * trunk_fc2[i] + out_bias[0]	Matches DeepONet/CRONet branch-trunk product with learned scalar bias, producing [1,21,61,2].

Intermediate storage

The kernel declares branch and trunk intermediate tensors as local static fp16_t arrays: branch_conv1_pre, branch_conv1_silu, branch_conv2_pre, branch_conv2_silu, branch_maxpool, branch_aap, branch_rnn_last, branch_fc1_pre, branch_fc1_silu, branch_fc2, trunk_conv1_pre, trunk_conv1_silu, trunk_conv2_pre, trunk_conv2_silu, trunk_aap, trunk_fc1_pre, trunk_fc1_silu, and trunk_fc2. Because these tensors are not arguments and have no AXI interface pragmas, HLS maps them into registers, LUTRAM, BRAM, or URAM. That is the PL equivalent of the AIE paper retaining intermediate activations in AIE memory/Memory Tiles.

Architectural diagram

DDR/host boundary	PL kernel resident CRONet	DDR/host boundary
branch_input 12000 FP16 values	BranchNet local tensors + static weights TrunkNet local tensors + static weights Final multiply + scalar bias	output 2562 FP16 values
trunk_input 2562 FP16 values	No external weight port No external intermediate tensor port	golden compare in host/testbench

Why this matches the specification and AIE paper

The AIE paper uses a fully on-AIE array dataflow implementation; this project uses a fully in-PL HLS kernel. They are not the same hardware fabric, but the fairness rules are parallel: full network on the accelerator, persistent weights, no intermediate DRAM traffic, and external memory limited to reading inputs and writing outputs. The current PL/HLS architecture satisfies those rules at the kernel-interface level.

The current kernel is correctness-first. It materializes named intermediate arrays because that makes the debug path and resource behavior easy to audit. A later optimized kernel can fuse or stream some layers, but it must preserve the same external contract and provide a debug variant that can still emit layerwise checkpoints.

Compliance Matrix

Criterion	Source expectation	Implemented evidence	Status
Model shape	60x20 element grid; output [1,21,61,2] / 2562 values.	cronet_config.hpp sets NELX=60, NELY=20, NODE_H=21, NODE_W=61, DOF=2, OUT_NEURONS=2562.	Met
Precision declaration	HLS FP16 revision; AIE paper is BF16 and must be labeled separately.	fp16_t=half under Vitis HLS; report label is PL/HLS FP16.	Met with caveat
Layerwise validation	Named tensors must match exported golden tensors before final output comparison.	tb_half_layerwise.cpp + cronet_half_debug.cpp compare named half outputs to fp16_torch_f32.	Met
Weight residency	Weights on chip; no DDR weight traffic.	static const fp16_t generated arrays included in the kernel.	Met
Intermediate residency	Intermediate activations on chip; no DDR tensor spill.	All major tensors are local/static arrays inside cronet_kernel; no m_axi ports for intermediates.	Met
Boundary DDR only	External memory allowed only for inputs and final output.	Only branch_input, trunk_input, and output are m_axi ports.	Met
AIE paper comparison	AIE uses BF16 and Memory Tile/AIE-local storage.	PL/HLS uses FP16 and BRAM/URAM/register/LUTRAM storage.	Architecturally aligned; datatype differs

Known limits and next optimization direction

This implementation is complete as a correctness-first PL/HLS realization, but it is not yet a final performance-tuned architecture. The main optimization work should be done after preserving the current validation gates.

- Specialize remaining generic loops to fixed 60x20 constants so HLS does not infer overly conservative bounds, especially around AdaptiveAvgPool3D.
- Add pipelining, unrolling, and array partitioning layer by layer while rerunning HLS half C-sim and sw_emu after each change.
- Bank the largest FC weight arrays across BRAM/URAM to increase MAC parallelism without violating on-chip storage.
- Fuse Conv/SiLU/pooling stages after standalone validation to reduce materialized intermediate buffers.
- Introduce DATAFLOW between BranchNet, TrunkNet, and final combine only after the local memory architecture is stable.

Conclusion

Steps 4, 5, and 6 satisfy the core CRONet/AIE-alignment requirements for a fair PL/HLS implementation. Step 4 proves FP16 HLS layer semantics against golden tensors; Step 5 makes all weights persistent FPGA-resident ROM data; Step 6 wraps the full CRONet graph in a single PL kernel whose only DDR-facing data are branch input, trunk input, and final output. The main caveat is precision labeling: this implementation is FP16, while the AIE paper reports BF16.

Source notes

Source	Used for
CRONet_HLS_1to1_Network_Spec_FP16.md	FP16 precision policy, on-chip storage requirements, layer-accurate validation criteria, non-negotiable compatibility rules.
ref_sources/croNET_AIE-ML ENGINE.pdf	AIE benchmark requirements: full accelerator-resident inference, on-chip weights/intermediates, boundary-only DRAM access, BF16 precision.
ref_sources/croNET.pdf	Original CRONet branch/trunk architecture, material-density history, load/trunk input, displacement output semantics.
vitis-hls-CROnet source tree	Actual HLS half debug path, generated FP16 ROM headers, and cronet_kernel interface/intermediate-storage evidence.